

Deployment Plan

Digital Logic Simulation Suite
Render (Backend) + Vercel (Frontend)
Prepared: October 2, 2026

Code Changes Required

These changes prepare your project for deployment. No backend logic or frontend design is being modified - only URLs and configuration.

1. Create frontend/config.js

NEW FILE

What: A tiny file with just one line - your Render backend URL.

Why: Right now, your JS files have `http://127.0.0.1:8000` scattered in 3 different files. That is your local PC address - it will not work on the internet. This config file gives one central place to set the URL, so when you deploy, you change it once and everything works.

2. Edit frontend/circuit-viewer.js (line 11)

EDIT

What: Remove the hardcoded `API_BASE = "http://127.0.0.1:8000"` line.

Why: `API_BASE` will now come from `config.js` instead, so there is no duplicate.

3. Edit frontend/converter-engine.js (4 places)

EDIT

What: Replace the hosts arrays (that try localhost, 127.0.0.1, etc.) with just `[API_BASE]`.

Why: On the live site, there is no localhost. It should only call your Render server.

4. Edit frontend/app.js (line 11)

EDIT

What: Change `"http://127.0.0.1:8000/convert"` to use the `API_BASE` variable instead.

Why: Same reason - point to your live backend instead of localhost.

5. Add config.js script tag to all HTML files

EDIT

What: Add one `<script src="config.js">` tag before the other scripts in each HTML page (`index.html`, `adder-subtractor.html`, `comparator.html`, `multiplexer.html`, `schematic.html`, and all converter HTML pages).

Why: So every page loads the `API_BASE` variable before the other JS files try to use it.

6. Fix requirements.txt

EDIT

What: Replace the current contents with just fastapi and uvicorn[standard].

Why: *Your backend runs on FastAPI + Uvicorn, but these are missing from your requirements file. Meanwhile, packages like scikit-learn, pandas, matplotlib are listed but not used by the backend - they would just slow down deployment.*

7. Create backend/__init__.py (empty file)

NEW FILE

What: An empty Python file - zero lines of code.

Why: *This tells Python that backend/ is a proper package. Without it, the imports inside main.py may fail on the Render server.*

8. Create .gitignore

NEW FILE

What: A file telling Git to ignore __pycache__/, server.log, .pytest_cache/, etc.

Why: *These are local junk files that should not be uploaded to GitHub or deployed.*

What is NOT Changing

- No backend Python logic changes (all .py files stay the same)
- No frontend UI or design changes (HTML/CSS unchanged)
- No features added or removed
- Local development still works (just change config.js back to localhost)

Deploy Backend on Render

1. Push code to GitHub

Run git add, commit, and push to your main branch.

2. Sign up on Render

Go to render.com and sign up with your GitHub account (free).

3. Create a Web Service

Dashboard > "New +" > "Web Service" > Connect your logic_sim_suite repo.

4. Configure settings

See the settings table below.

5. Click "Create Web Service"

Wait 2-3 minutes for it to build and deploy.

6. Copy your URL

You will get something like: https://logic-sim-backend.onrender.com

7. Test it

Open your-url/docs in browser - you should see the FastAPI Swagger UI.

Render Settings:

Name	logic-sim-backend	
Region	Singapore (or closest to you)	
Branch	main	
Runtime	Python	
Build Command	pip install -r requirements.txt	
Start Command	cd backend && uvicorn main:app --host 0.0.0.0 --port \$PORT	
Instance Type	Free	

Deploy Frontend on Vercel

1. Update config.js

Put your real Render URL in frontend/config.js, then push to GitHub.

2. Sign up on Vercel

Go to vercel.com and sign up with your GitHub account (free).

3. Import your project

"Add New..." > "Project" > Select your logic_sim_suite repo.

4. Configure settings

See the settings table below.

5. Click "Deploy"

Your site will be live in about 30 seconds!

Vercel Settings:

Framework Preset	Other	
Root Directory	frontend (click	Edit to change)
Build Command	(leave empty)	
Output Directory	.	

Verification Checklist

- ☐ Open your Vercel URL - Homepage loads with all 4 sections
- ☐ Click "Binary to/from Gray Code" - Simulator page loads
- ☐ Enter bits and click Convert - Result appears
- ☐ Click "Schematic" - KiCad viewer loads with interactive circuit
- ☐ Toggle bits on the schematic - Circuit updates live
- ☐ Open Render URL/docs - FastAPI Swagger UI shows all endpoints
- ☐ Test comparator simulator - Results displayed correctly
- ☐ Test adder/subtractor simulator - Addition and subtraction work

Free Tier Limitations

Render (Free):

- Spins down after 15 minutes of inactivity. First request after sleep takes ~30 seconds.
- 750 hours/month - more than enough for project demos.

Vercel (Free):

- 100 GB bandwidth/month - more than enough for a student project.
- No server-side code - fine, your frontend is pure static files.

Good news: Your app already handles Render cold-starts gracefully! The local fallback engines in converter-engine.js and circuit-viewer.js provide instant results in the browser while the backend wakes up.